

Lab 7 Linear Regression (Tutor Copy)

Learning Outcome

1. Describe the assumptions and process of regression analysis.
 2. Apply regression models on data variables.
 3. Interpret the outcomes of regression models for data-driven decisions.
 4. Recognize how data analytics is used in Smart Manufacturing environment.
-
- Able to explain that machine learning is a subset of artificial intelligence where computers learn from data to make decisions or predictions.
 - Able to contrast traditional programming (explicit instructions) with machine learning (learning from data).
 - Know the key terms such as algorithm, model, training, and dataset
 - Understand the workflow and nodes used in KNIME for machine learning
 - To use the **Linear Regression Learner** to generate the model
 - To use **Regression Predictor** to validate the model
 - To use **Numeric Scorer** to determine the root mean square error

Exercise 1

Data file: *CO2_machineLearning.csv*

Start a **new Workflow** and give the name “*CO2_machineLearning*”

Add **CSV Reader** node and configure to read the file *CO2_machineLearning.csv*
Execute the node.

Write the information about the data:

Number of observations = 194

Number of variables (excluding Date and Time) = 3

What does each row represent? Hourly reading of CO2, Temperature and Relative Humidity

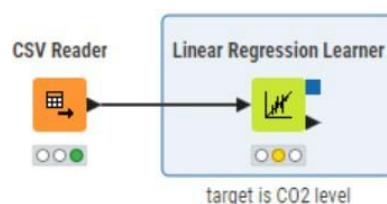
In this machine learning, our target variable is CO2 Level

Our independent variables are: Temperature and Relative Humidity

Questions for machine learning:

How accurate is the Linear Regression model in predicting CO2 level based on our selected independent variables?

Add a **Linear Regression Learner**.



Configure the Target to be “CO2” and the independent variables are Temperature and Relative Humidity %.

File

Settings Flow Variables Memory Policy

Target
D CO2 (ppm) 

Values
☒ Manual Selection ☐ Wildcard/Regex Selection

Exclude
Filter
S Date and Time
☒ Enforce exclusion

Include
Filter
D Temperature (°C)
D Relative Humidity (%) 
☐ Enforce inclusion

Regression Properties
☐ Predefined Offset Value: 0

Missing Values in Input Data
☐ Ignore rows with missing values.
☒ Fail on observing missing values.

Scatter Plot View
First Row: 1
Row Count: 20,000

OK Apply Cancel ?

Execute and Open Views.



Linear regression attempts to model the relationship between two variables by fitting a linear equation to observed data. One variable is the dependent (target) variable while one or more variable(s) is/are the explanatory (input) variable(s). For example, a modeller might want to relate the weights of individuals to their heights using a linear regression model.

Before attempting to fit a linear model to observed data, a modeller should first determine whether there is a relationship between the variables of interest. This does not necessarily imply that one variable causes the other (for example, higher SAT scores do not cause higher college grades), but that there is some significant association between the two variables.

A linear regression line has an equation of the form $Y = a + bX$, where X is the explanatory variable and Y is the dependent variable. The slope of the line is b , and a is the intercept (the value of y when $x = 0$).

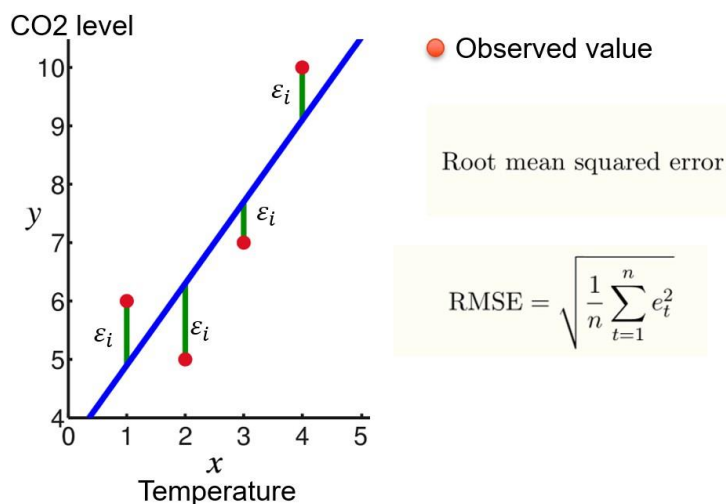


How does the Learner work?

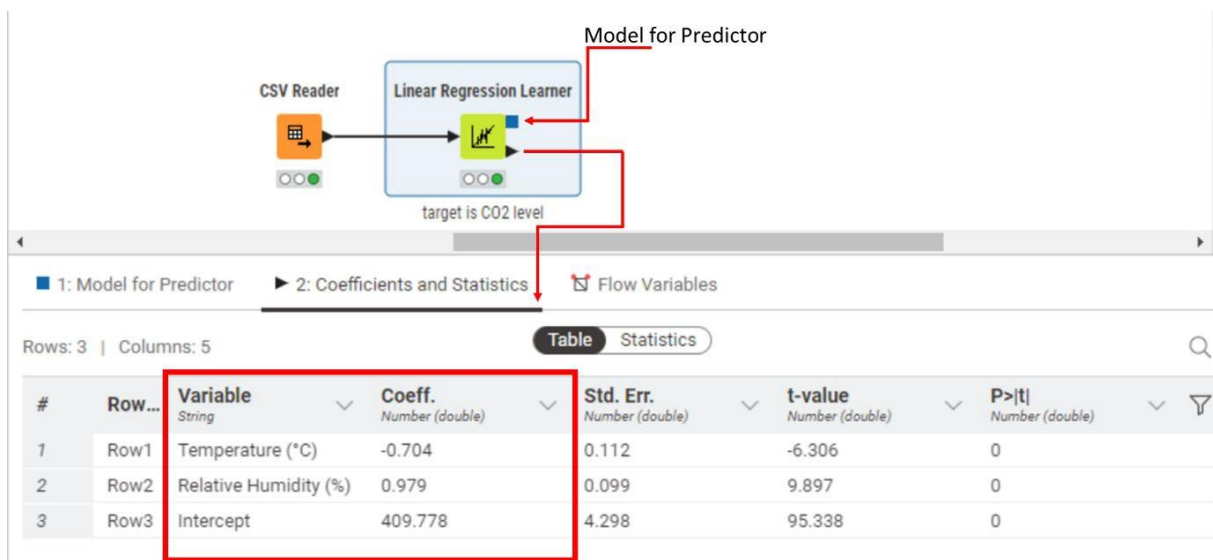
- Linear Regression Model with **multiple independent variables**:

$$Y = \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \dots + \beta_0$$

Learner will determine a trend line that passes through all/most of the observation points, and check the Root Mean Square Error.



It will converge to a line with the least RMSE.



$$Y = -0.704 \cdot T + 0.979 \cdot RH + 409.778$$

This is the linear regression model generated by the Learner to predict the CO2 (Y-value) from temperature (T) and RH (relative humidity %) based on the given data.

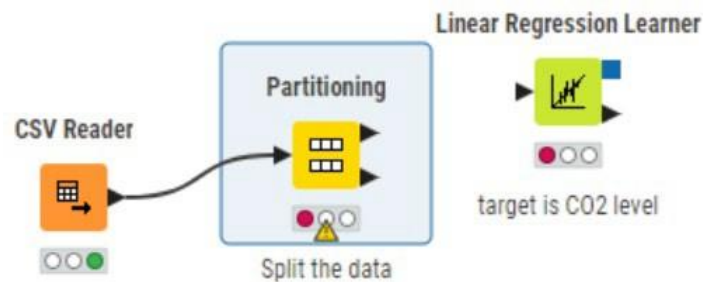
This is the best fit line for the given set of data.



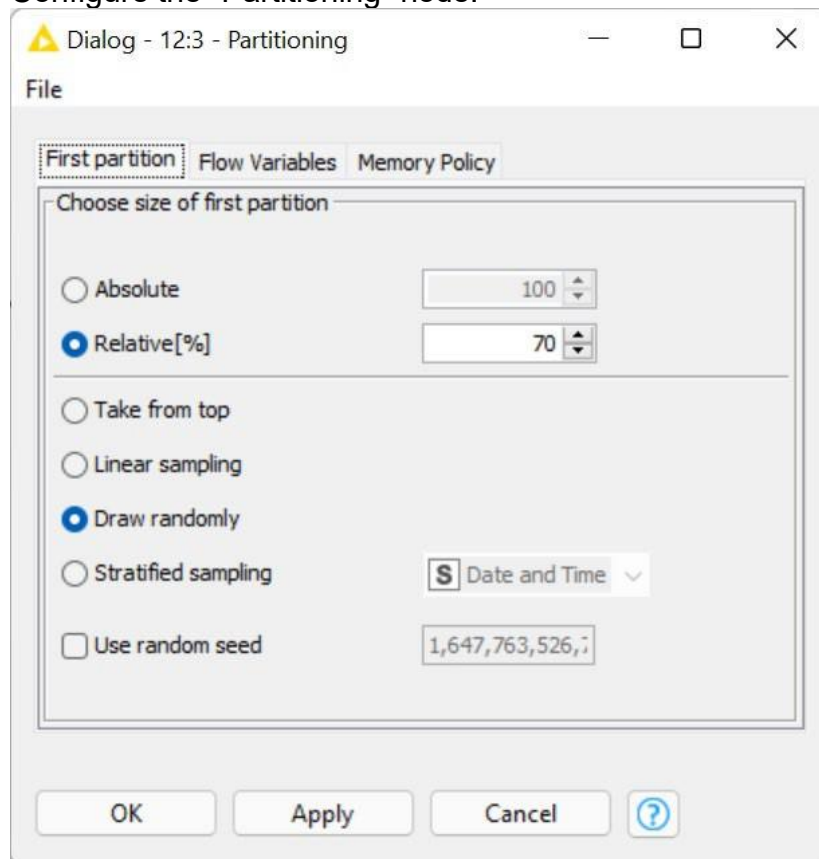
How accurate is the model when tested with “new” data it has not learned before?

To validate the model, we will split the data into **Training** set and **Test** set.

Add a **Partitioning** node before the Learner.



Configure the “Partitioning” node:



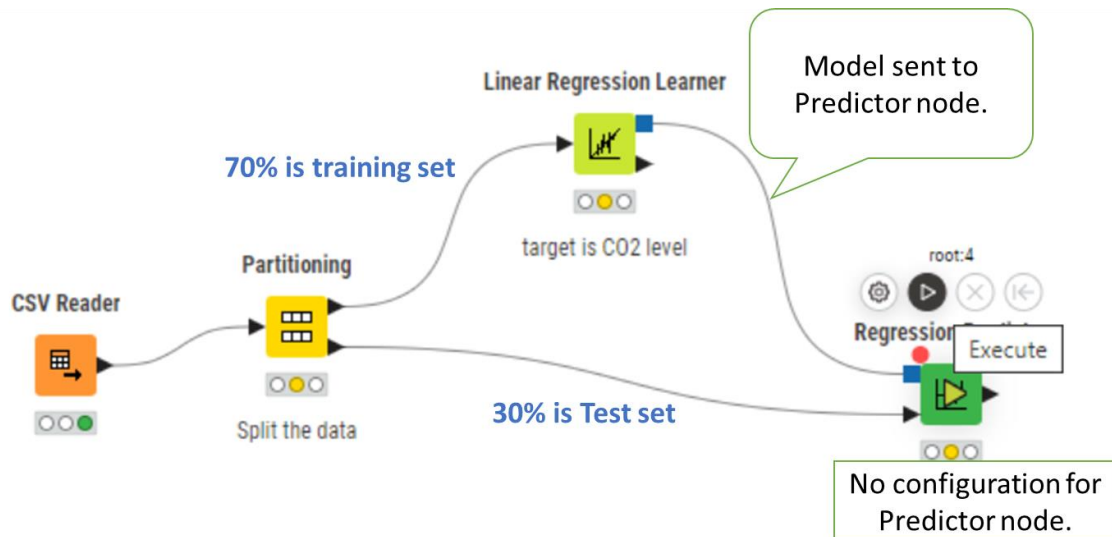
Divide the data into 70% and 30%.

70% will be the training set used by **Linear Regression Learner**
(which is $0.7 \times 194 = 135.8$ round to 135)

30% will be the test set for **Regression Predictor**.
(which is $194 - 135 = 59$)

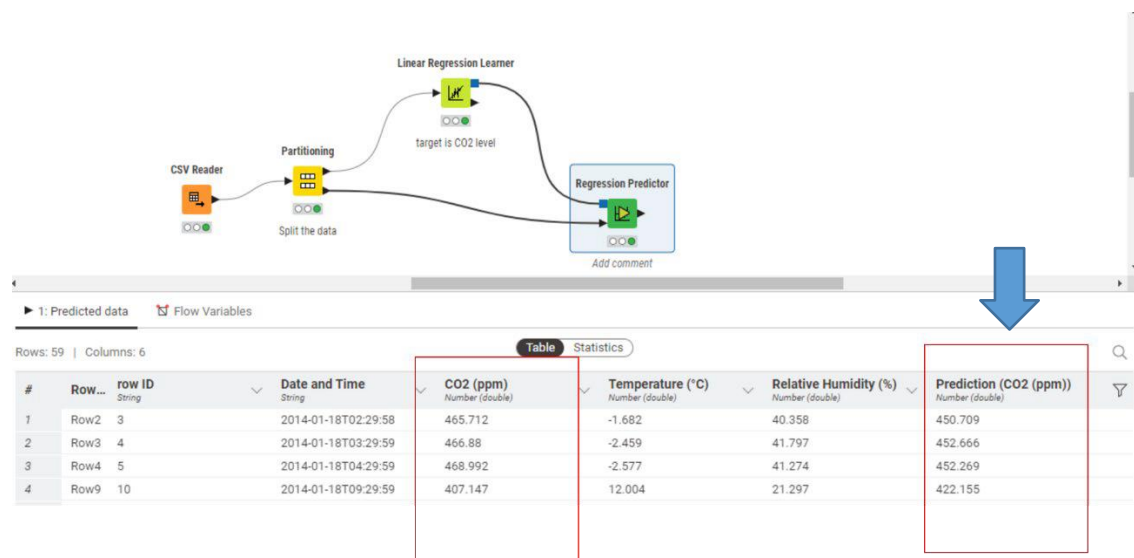
Add a **Regression Predictor** node:

There is nothing to configure for predictor node.



The predictor is using the algorithm (which is also known as the Model) generated by the **Regression Learner** but work on a different set of data (partition 2, 30%)

Execute the **Regression Predictor** node.



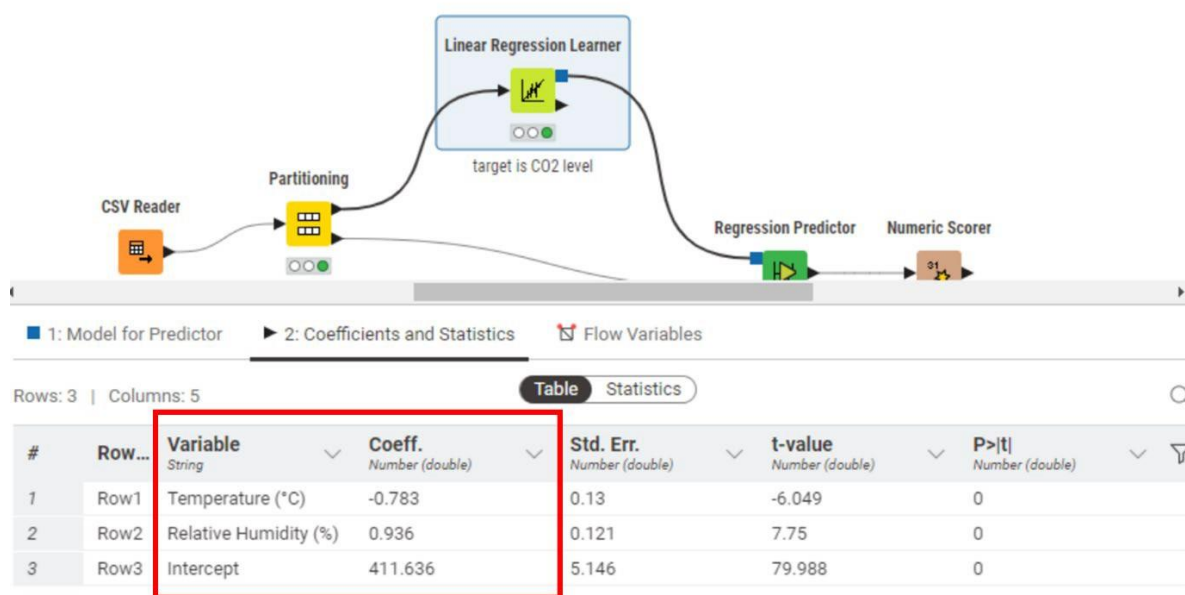
New column added. It is the predicted value.



Note that this set of data has 59 rows. Why? **Test data set is 59.**

The predictor uses the second partition (30% data) to predict CO2.
The partitioning node split the data so that the predictor uses different set of data from the Learner.

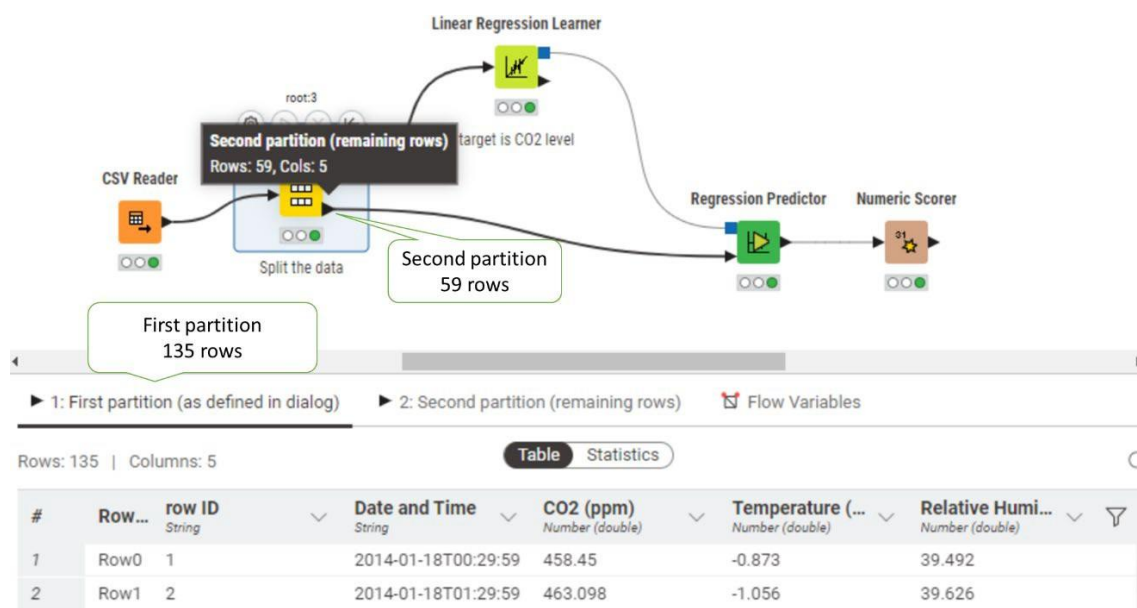
Check the coefficients and Statistics of the Learner:



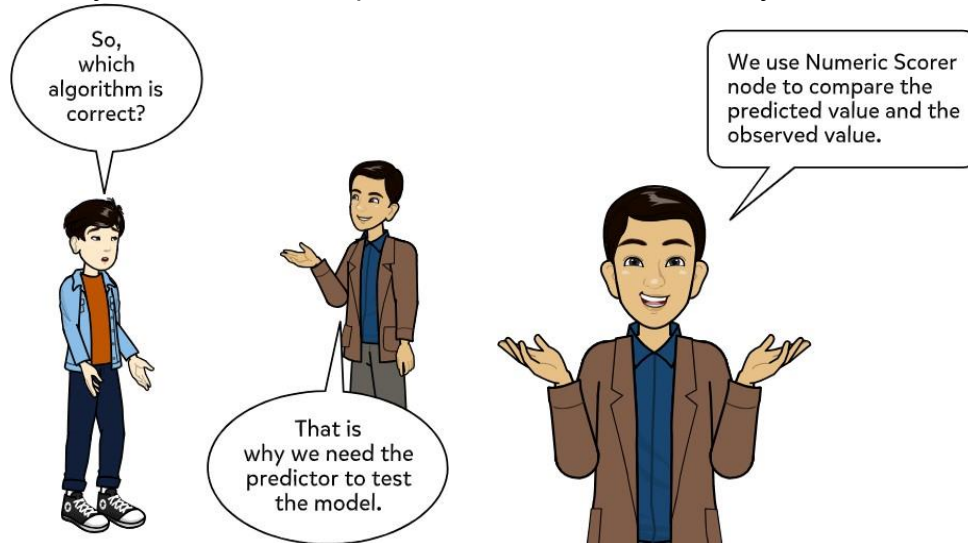
The algorithm proposed is $Y = -0.783 \cdot T + 0.936 \cdot RH + 411.636$

This is different from previous output (pg 3). It may also be different from your output!

This is the algorithm generated by the learner base on my 70% training set (135 observations).

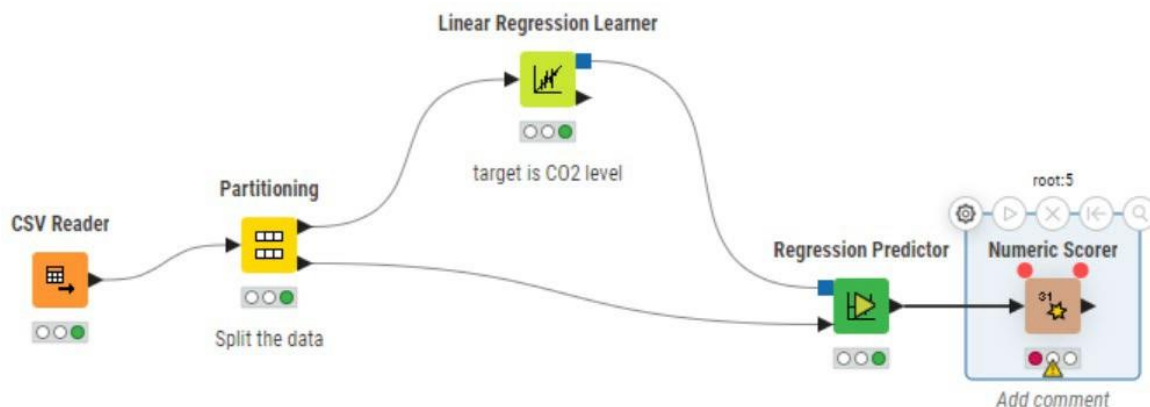


My 70% of data is **randomly** selected from 194 rows. Although the 194 rows are the same as yours, but the first partition set is different from yours.

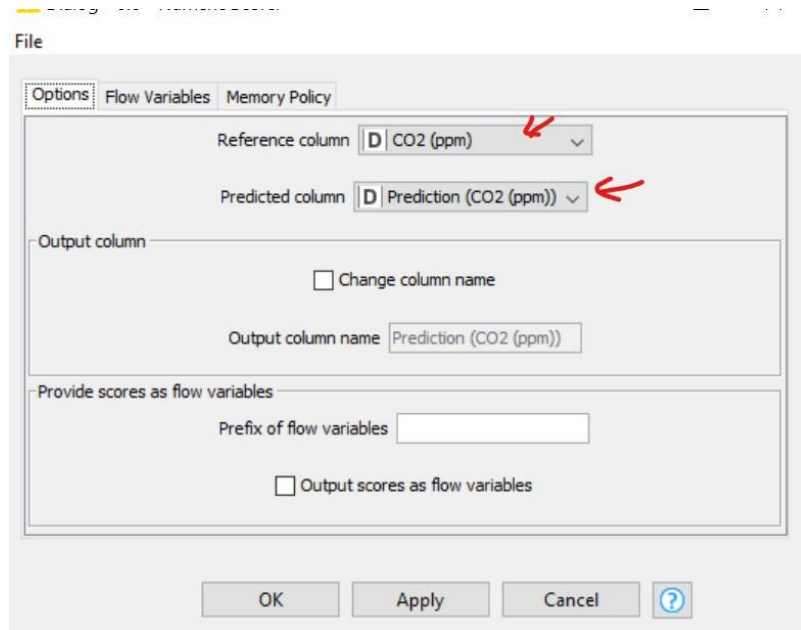


Comparing the predicted value and the observed value, what is the error?

Add a **Numeric Scorer** node.



Configure the **Numeric Scorer**:



Execute and Open Views (Numeric Scorer node)

Statistic...	
R²:	0.639
Mean absolute error:	10.819
Mean squared error:	170.595
Root mean squared error:	13.061
Mean signed difference:	-1.591
Mean absolute percentage error:	0.025
Adjusted R²:	0.639

**** Note that you may not get the same statistics result as mine because the 59 observations is randomly picked from the 194. Though we may use the same data set, but the 59 observations are not the same.**

R^2 (R-squared) is a statistical measure of how close the data are to the fitted regression line. It is also known as the **coefficient of determination**. It is sometimes expressed as a percentage. R^2 value ranges from 0% to 100% (or 0.0 to 1.0 as in the case in KNIME)

$$R^2 = \frac{\text{Variance explained by the model}}{\text{Total variance}}$$

The larger the R-squared value (the lower the root mean squared error) the more precisely the **selected independent variables** can predict the value of the response variable (target).

**Discussion**

A regression model that could produce accurate prediction should have... (delete away “high” or “low” for each case)

High / ~~Low~~ R^2

High / ~~Low~~ mean absolute error

High / ~~Low~~ mean square error

High / ~~Low~~ root mean squared error

High / ~~Low~~ mean signed error

Conclusion for this data set:

The Root Mean Squared Error (RMSE) is 13.061.

The R-squared value (also known as coefficient of determination) shows 0.639. It means 63.9% of the data fit well to the regression line.

Linear Regression model can predict the CO₂ level with moderate accuracy.

****Note that in your project, you will attempt to change the partitioning %, or methods of random selection to achieve the lowest RMSE or highest R-squared.**

Exercise 2 (Some challenge)

Data file: [laptopPrice.csv](#)

For our task this week, we make use of a dataset containing a list of laptop prices for 823 laptops. For more details, you can visit this link ([Laptop Prices Dataset | Kaggle](#)).

Specific questions that we would like to answer in this session:

- Which are the important features (key predictor) that are correlated a laptop's price?
- How accurate is our model to predict the laptop's price?

Data dictionary:

Variable name	Data type
Company	String
Processor_brand	String
Processor_name	String
Processor Generation	String
Ram_gb	String
Ram_type	String
SSD size	String
HDD size	String
OS type	String
OS bits	String
Graphic_card_gb	String
weight	String
warranty	String
Touchscreen	String
msoffice	String
Price_rupee	Number(integer)
Rating	String
Number of ratings	Number(integer)
Number of reviews	Number(integer)

Which variable is our target? Price rupee

Which variable can we use as independent variables? Numerical Data Types

The challenge in this data set is there are only 2 numerical variables to use to predict "Price". How can we make use of the STRING variable type?

Data Transformation

Notice these few variables (in the figure below) consist of number and text in one cell; The text are just the units, like GB, bit and stars.

ram_gb String	ssd String	hdd String	os_bit String	graphic_card_gb String	rating String
4 GB	0 GB	1024 GB	64-bit	0 GB	2 stars
4 GB	0 GB	1024 GB	64-bit	0 GB	3 stars
4 GB	0 GB	1024 GB	64-bit	0 GB	3 stars
8 GB	512 GB	0 GB	32-bit	2 GB	3 stars
4 GB	0 GB	512 GB	64-bit	0 GB	3 stars
4 GB	0 GB	512 GB	64-bit	0 GB	3 stars
4 GB	0 GB	512 GB	32-bit	0 GB	3 stars
4 GB	0 GB	1024 GB	32-bit	2 GB	3 stars
8 GB	0 GB	1024 GB	32-bit	0 GB	3 stars
4 GB	0 GB	512 GB	32-bit	4 GB	4 stars
4 GB	0 GB	512 GB	32-bit	6 GB	4 stars

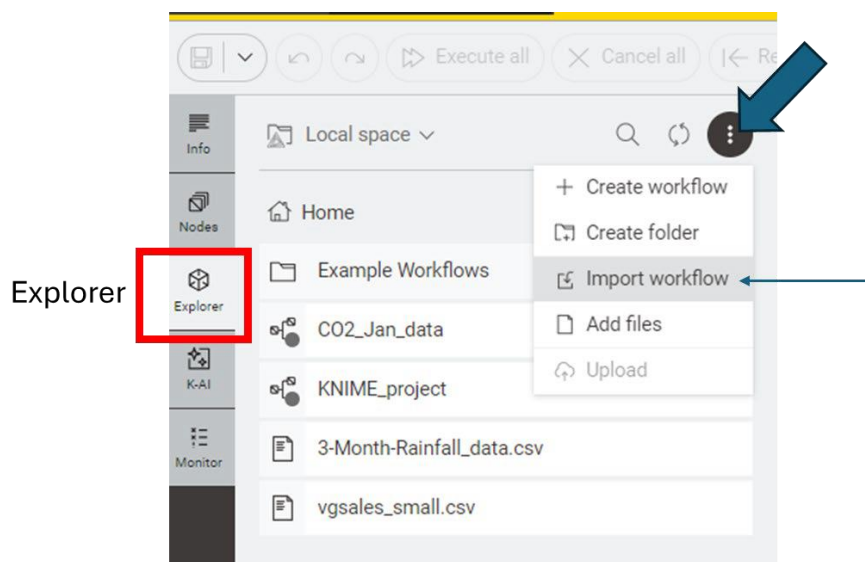
The number and text is separated by a “space” or “-”.

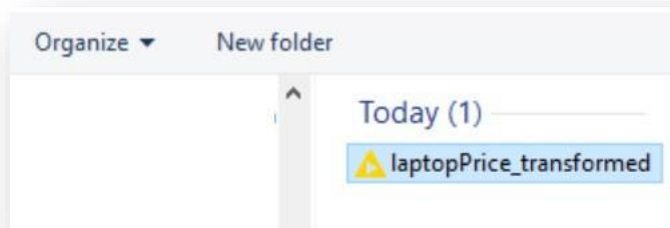
We can use **Cell Splitter** node to easily extract out the numbers. Refer to KNIME workbook on “Cell Splitter” explanation.(pg 37)

The data transformation is already done in the file: [laptopPrice_transformed.knwf](#)
It is the KNIME Export file.

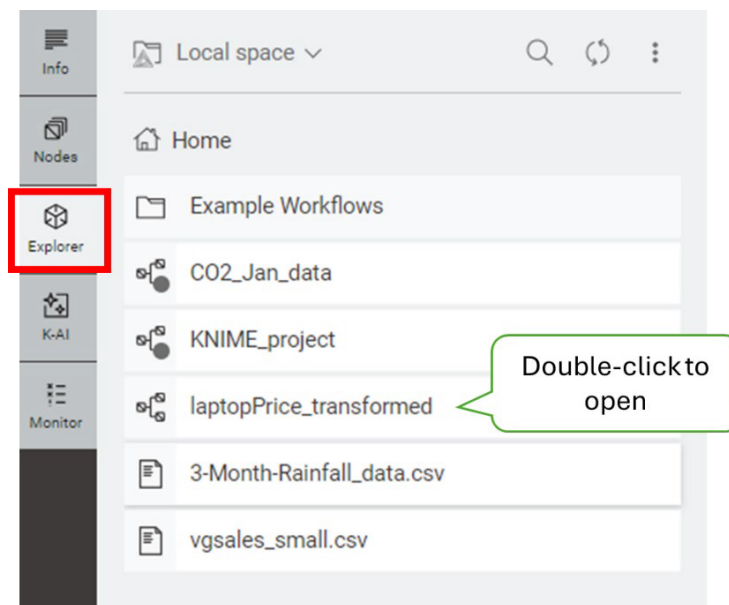


How to read in the KNIME exported file?

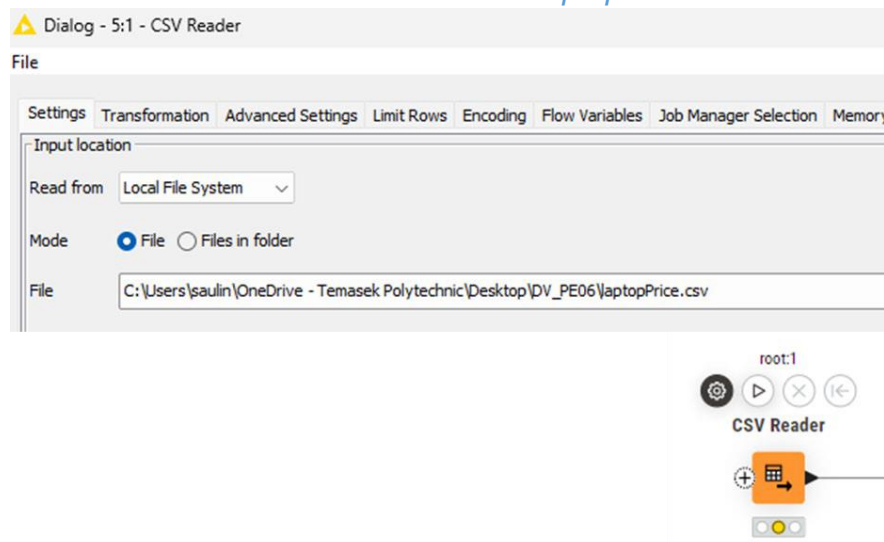




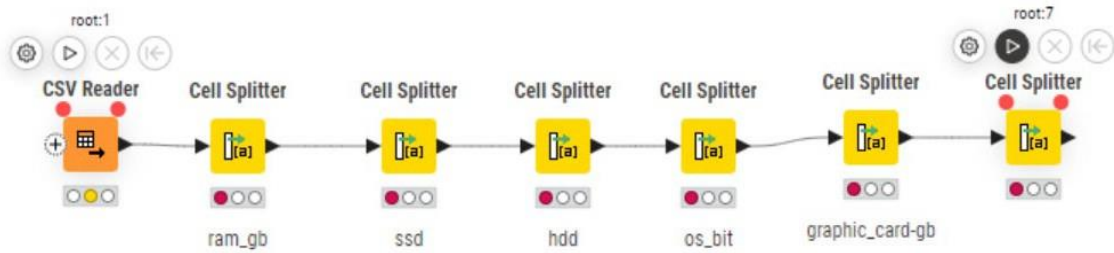
Browse to select the KNIME file.



You will need to read in the CSV file: [laptopPrice.csv](#)



You can go to the last node to “execute”



All node status will turn “green”.
Click on the last Cell Splitter. It should reflect 31 columns.

1: Output Table Flow Variables

Rows: 823 Columns: 31

#	RowID	brand	processo...	processo...	processo...	ram_gb	ram_type	ssd
1	Row0	ASUS	Intel	Core i3	10th	4 GB	DDR4	0 GB
2	Row1	Lenovo	Intel	Core i3	10th	4 GB	DDR4	0 GB

Data Exploration (linear correlation)

You can add a Linear correlation node (after the last Cell Splitter node) to check which variables are highly correlated with **“Price”**.

Exclude all String data type.

Dialog - - Linear Correlation

File

Options | Flow Variables | Job Manager Selection | Memory Policy

☒ Manual Selection ☐ Wildcard/Regex Selection

Exclude

Filter

- S ram_type
- S ssd
- S hdd
- S os
- S os_bit
- S graphic_card_gb
- S weight
- S warranty
- S Touchscreen
- S msoffice

☒ Enforce exclusion

Include

Filter

- I Price
- I Number of Ratings
- I Number of Reviews
- I ram_gb_Arr[0]
- I ssd_Arr[0]
- I hdd_Arr[0]
- I os_bit_Arr[0]
- I graphic_card_gb_Arr[0]
- I rating_Arr[0]

☐ Enforce inclusion

Output column pairs

☐ Include all column pairs

☒ Include only column pairs of compatible columns

☐ Include only column pairs with a valid correlation

Possible Values Count 50

p-value

☒ two-sided

☐ one-sided (right)

☐ one-sided (left)

OK Apply Cancel ?

Linear Correlation Node Configuration

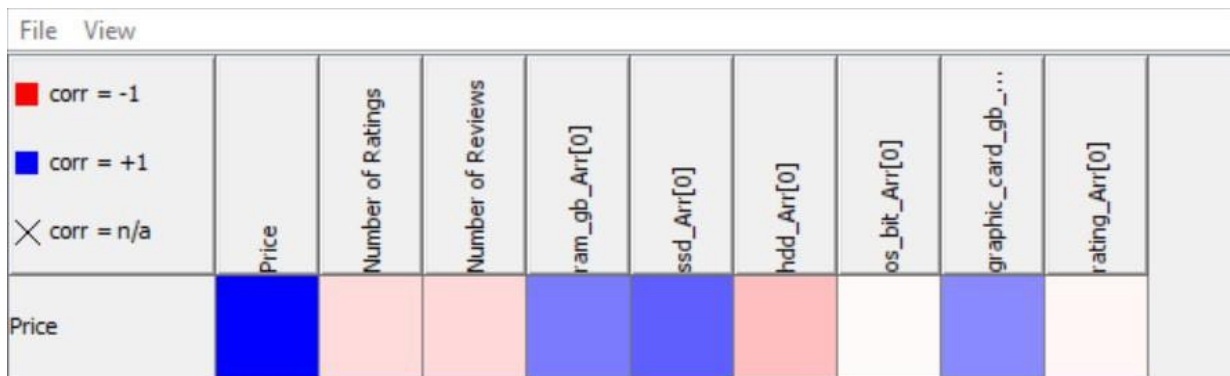


Why do we want to find the linear correlation?

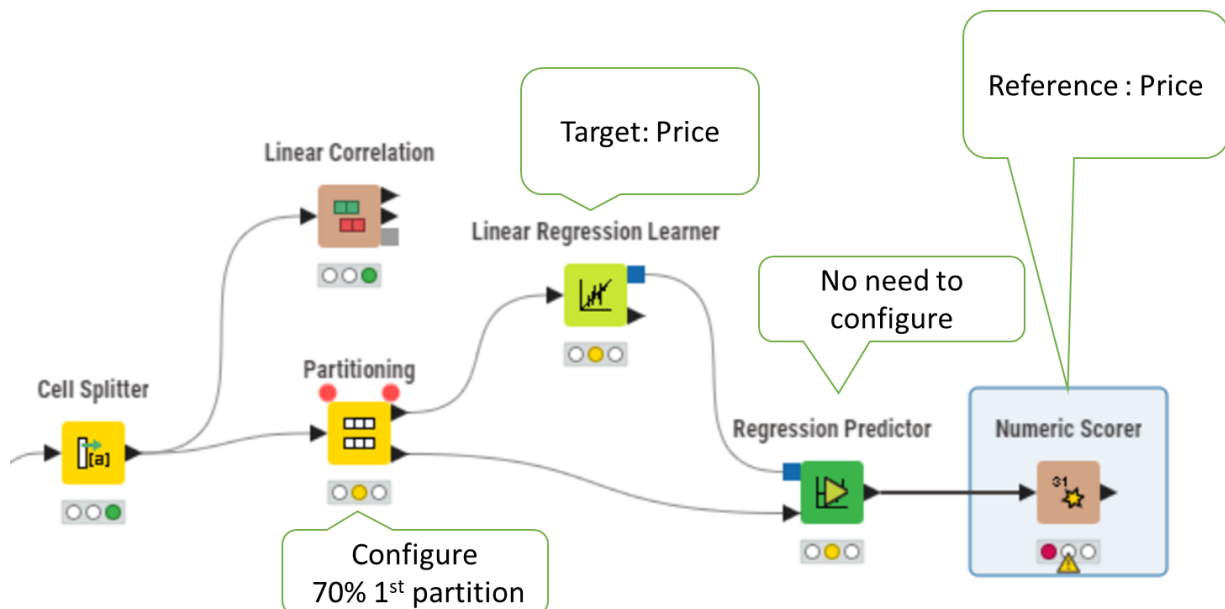
To determine which variables are highly correlated to the laptop price.

Execute the linear correlation node. Check out the correlation matrix.

Which variable are highly correlated with “Price”?



Having some idea how the variables are correlated with Price, we will setup the machine learning workflow.



Add the necessary nodes for machine learning

Refer to next figure to configure the **Learner**.

Configure Learner:

Dialog - 5:10 - Linear Regression Learner

File

Settings Flow Variables Job Manager Selection Memory Policy

Target: Price

Values

Manual Selection Wildcard/Regex Selection

Exclude

Filter

rating
ram_gb_Arr[1]
ssd_Arr[1]
hdd_Arr[1]
os_bit_Arr[0]
os_bit_Arr[1]
graphic_card_gb_Arr[1]
rating_Arr[0]
rating_Arr[1]

Enforce exclusion

Include

Filter

Number of Ratings
Number of Reviews
ram_gb_Arr[0]
ssd_Arr[0]
hdd_Arr[0]
graphic_card_gb_Arr[0]

Enforce inclusion

Regression Properties

Predefined Offset Value: 0

Missing Values in Input Data

Ignore rows with missing values.
Fail on observing missing values.

Scatter Plot View

First Row: 1
Row Count: 20,000

OK Apply Cancel ?

Target is "Price"

Include only numerical data type. Select those that are highly correlated to "Price".

Configure Numeric Scorer:

Dialog - 5:12 - Numeric Scorer

File

Options Flow Variables Job Manager Selection Memory Policy

Reference column: Price

Predicted column: Prediction (Price)

Output column

Change column name

Output column name: Prediction (Price)

Provide scores as flow variables

Prefix of flow variables

Output scores as flow variables

Adjusted R squared

Number of predictors: 0

OK Apply Cancel ?

Execute Numeric Scorer.

Statistics on Linear Regression				
Variable	Coeff.	Std. Err.	t-value	P> t
Number of Ratings	9.7613	8.6488	1.1286	0.2595
Number of Reviews	-101.298	73.3689	-1.3807	0.1679
ram_gb_Arr[0]	3,115.935	332.1048	9.3824	0.0
ssd_Arr[0]	64.428	6.1147	10.5366	0.0
hdd_Arr[0]	9.7492	4.0483	2.4082	0.0163
graphic_card_gb_Arr[0]	5,073.0478	698.7249	7.2604	1.27E-12
Intercept	12,826.5014	3,883.8247	3.3025	0.001

R-Squared: 0.5109
Adjusted R-Squared: 0.5057

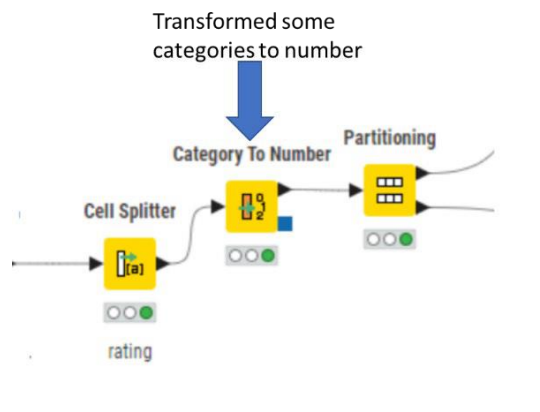
Statistics - 4:11 ...	
R ² :	0.599
Mean absolute error:	19,664.313
Mean squared error:	868,205,889.377
Root mean squared error:	29,465.334
Mean signed difference:	-730.449
Mean absolute percentage error:	0.241
Adjusted R ² :	0.599

R^2 shows 59.9% of the variability observed in the target variable is explained by the regression model. Fit well in the regression model.

Can we add more numerical data type to improve the accuracy of the model?

Transformed Category to Number

We can transform some STRING variable to number using **Category to Number** node.



Configure Category to Number:

Dialog - Category To Number

File

Columns to transform | Flow Variables | Job Manager Selection | Memory Policy

☒ Manual Selection ☐ Wildcard/Regex Selection

Exclude

Filter

- S processor_brand
- S processor_name
- S processor_gnrtn
- S ram_gb
- S ram_type
- S ssd
- S hdd
- S os

☒ Enforce exclusion

Include

Filter

- S brand
- S weight
- S Touchscreen

☐ Enforce inclusion

☒ Append columns

Column suffix: (to number)

Start value: 0

Increment: 1

Max. categories: 100

Default value:

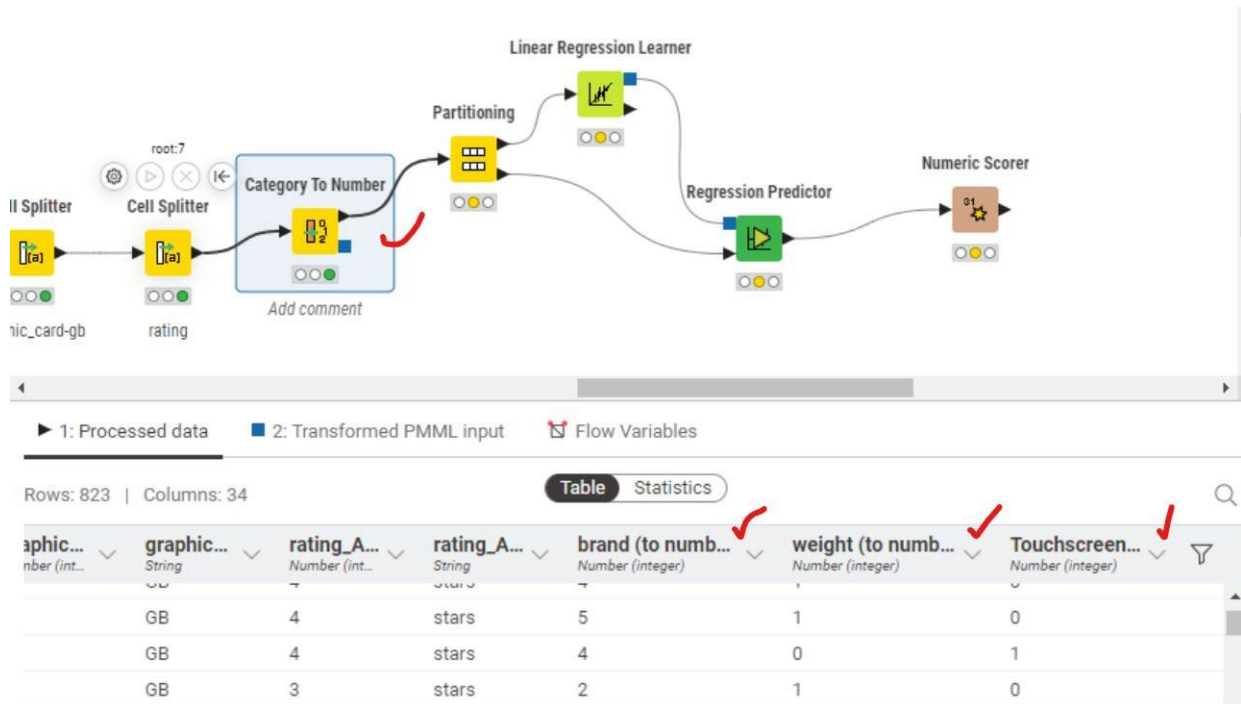
Map missing to:

OK Apply Cancel ?

Selected 3 String variables which I think are correlated to price, to convert to number

This function will assign a “number” 1 to 100 to each category. Our data set should have less than 100 values in each category.

Execute the node.



3 additional columns

Check the configuration at the **Learner**:

The 3 additional numeric data types are automatically added to the model

Execute the Numeric Scorer Node to check the R-squared:

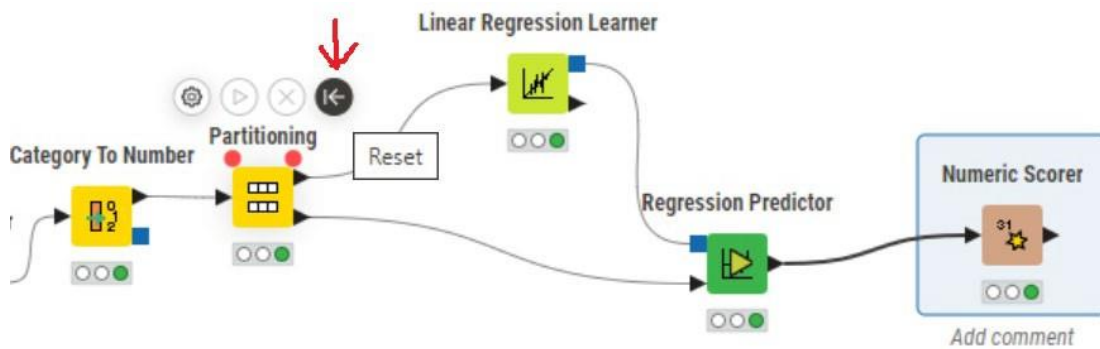
R ² :	0.49
Mean absolute error:	19,118.266
Mean squared error:	798,585,683.53
Root mean squared error:	28,259.258
Mean signed difference:	197.388
Mean absolute percentage error:	0.238
Adjusted R ² :	0.49

Notice R-squared drops.

(Or, yours is higher?)

The more complex the model (more variables) may not lead to higher accurate predictions.

What about “reset” the **partitioning node**?



This means, the program will generate another random set of training and test set.

Notice you will also get different result from **Learner's** model and **Numeric Scorer**.

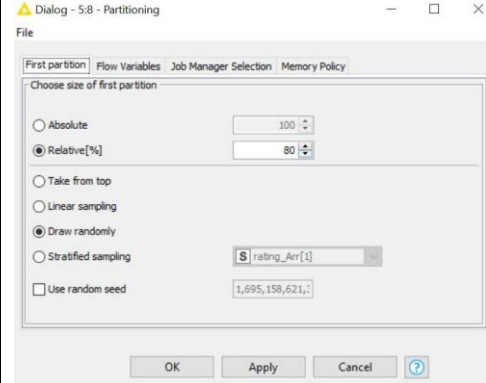
Therefore, there is no standard answer to a machine learning model. It depends on the data set.

The objective in machine learning is to

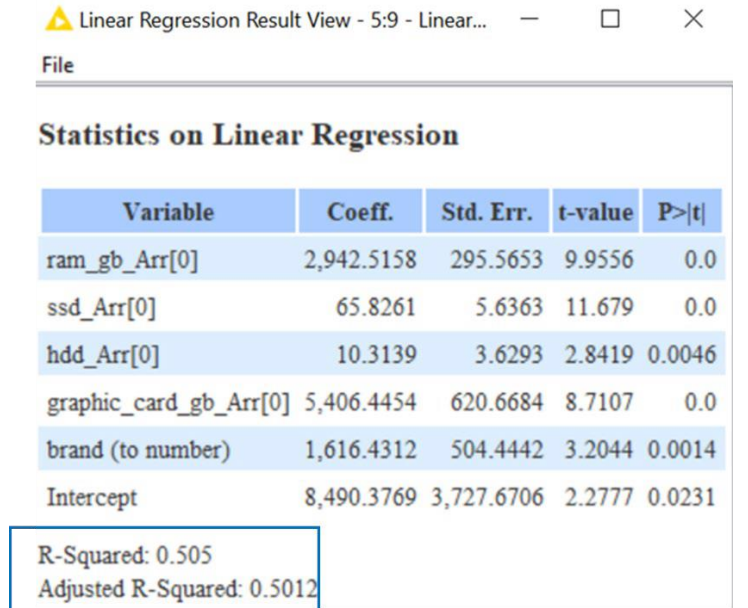
- change the partition ratio or
- select variables that are highly correlated with the target in the Learner,

to obtain the best R-squared.

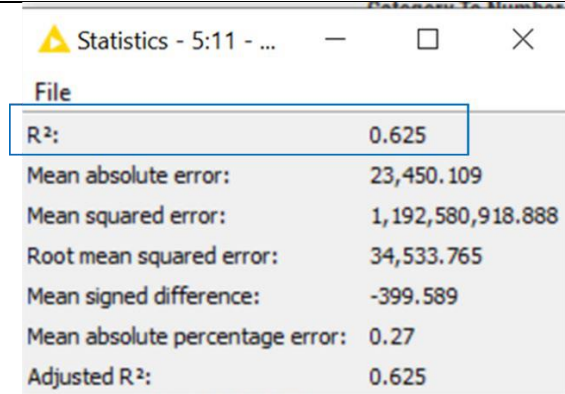
After a few tries, this is my best result:



Partitioning Node



Regression Learner Node



Numeric Scorer node

Are you able to get a better R-squared result than mine?

NOTE: In your project, you should screen shot partitioning node, regression result and numeric scorer result as one “answer” to your model.



Do you think this is the most accurate model?

Notice the R-squared value in Regression Learn shows 0.5 and R-squared value in Numeric Scorer shows 0.626.

They are the R-squared for the training set and test set respectively.

It does not mean this model (or the algorithm) will predict equally accurate result with similar R-squared value with a new set of data.

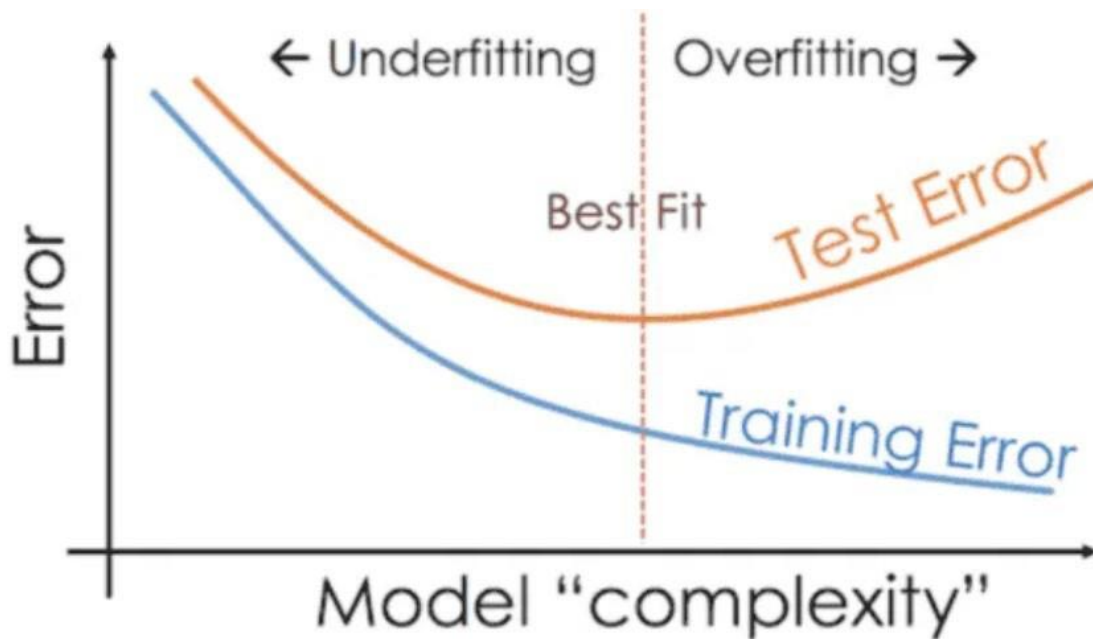


Beware of over-fitting

Does it mean more independent variables, and R-squared value is very high, the prediction is very accurate?

When the model becomes more complex (using a lot of independent variables), some variable used may be “noises” that are not accurate.

When we run the model on a new (“unseen”) data set, the R-squared will be very low (< 0.5). This is the result of overfitting.



[Introduction to Supervised Machine Learning - Developer Support \(microsoft.com\)](https://docs.microsoft.com/en-us/machine-learning-and-azure/supervised-machine-learning/underfitting-overfitting)

[Underfitting vs. Overfitting \(vs. Best Fitting\) in Machine Learning | by Gaurav Rajpal | Analytics Vidhya | Medium](https://medium.com/@gauravrajpal/underfitting-vs-overfitting-vs-best-fitting-in-machine-learning-1e1e1e1e1e1e)

Project (Section 9)

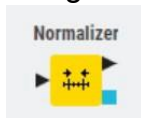
Decide on your target.

Decide on your learner:

If your target is a numerical data type, you can use **Linear Regression Learner**.
But, if your target is categorical, eg. Yes or No, High or Low, Pass or fail
Then, you should use **Logistic Regression Learner**, that we will cover in our next lab session.

Decide if you need to do data transformation.

If the variables are of different scale/units, you may want to perform “Normalisation” using the **Normalizer node**.



You may not use this node, if the numeric scorer result is quite good already.

Are there categorical data type that you think correlated with your target?
You can use “**Category to Number**” node to assign a random number to each type.

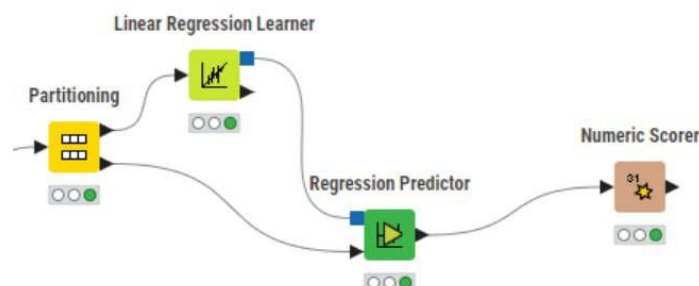
Do you need to use Cell Splitter?

Unless you have a string of information in a cell that you want to extract some number/info from it, otherwise, there is no need to use Cell Splitter.



Do you need to use **Rule Engine** node to assign “special” name to group of values? This node is like “grouping” in TABLEAU. There is no need to use this node.

The workflow setup is the same, starting with “**Partitioning**” node.



But take note which data table you should pass to Partitioning node.
It should be clean data (output from Missing Value node), or after data transformation steps.